Adversarial Search & Game Playing

Games · Game Trees · Minimax · Multiplayer Games

Course No: 0613-401

Course Title: Artificial Intelligence

Ekramul Haque Amin

Lecturer

Department of Artificial Intelligence and Data Science

Dhaka International University (DIU)

Today’s Content

- Recap: search when we are the only agent
- Games in AI and types of games
- A game as a search problem; game trees
- The Minimax algorithm — step by step
- Minimax on Tic-Tac-Toe
- Properties of Minimax
- Optimal decisions in multiplayer games

Recap: From Single-Agent Search to Games

- So far (BFS, A*, Hill Climbing, CSP) the agent was alone: the environment did not fight back.
- In a game, another agent (the opponent) chooses moves to make us lose.
- We cannot plan a fixed sequence of moves — we need a strategy: a move for every possible reply.
- Searching in such competitive multi-agent environments is called adversarial search.
- Link to Lecture 2: games are multi-agent environments.

Why Games in AI?

- **Clear rules, clear winner — easy to state as a problem.**
- **Huge search spaces: chess has about 35 moves per position and games of about 100 moves → about $35^{100} \approx 10^{154}$ possible games.**
- **Milestones:**
  - **1997: IBM Deep Blue beats world chess champion Garry Kasparov.**
  - **2016: DeepMind's AlphaGo beats Go champion Lee Sedol.**
- **The same ideas are used in negotiation, security, economics and robot planning against opponents.**

Types of Games

	Deterministic (no chance)	Chance (dice, cards)
Perfect information (see everything)	Chess, Checkers, Go, Tic-Tac-Toe	Ludo, Backgammon, Monopoly
Imperfect information (hidden parts)	Battleship, blind Tic-Tac-Toe	Poker, Bridge, Scrabble

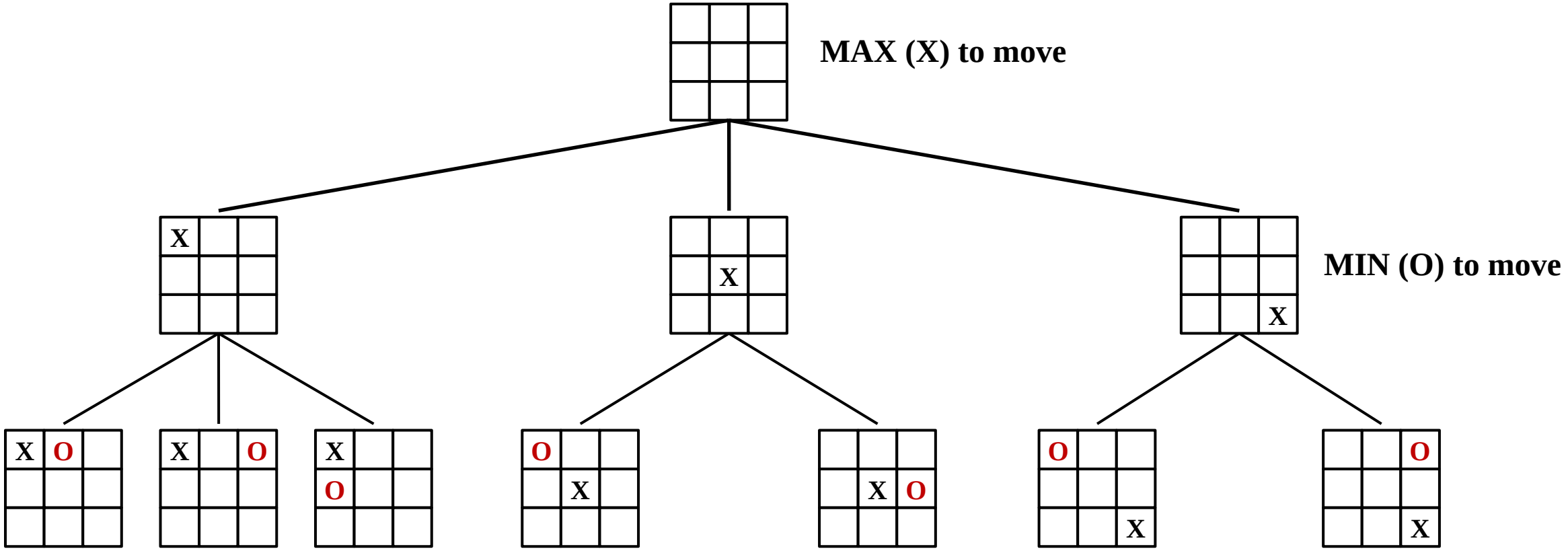
- Today: deterministic, perfect-information, two-player, zero-sum games.
- Zero-sum: one player's gain is exactly the other's loss (win +1, lose −1, draw 0).

A Game as a Search Problem

Part	Meaning	Tic-Tac-Toe
S0 — initial state	How the game starts	Empty 3 × 3 board
PLAYER(s)	Whose turn it is in state s	X (MAX) or O (MIN)
ACTIONS(s)	Legal moves in s	Any empty square
RESULT(s, a)	State after move a	Board with the new mark
TERMINAL-TEST(s)	Is the game over?	Three in a row, or board full
UTILITY(s, p)	Final score for player p	+1 X wins, 0 draw, −1 O wins

- The two players are called MAX (wants high utility) and MIN (wants low utility).

The Game Tree



… and so on, down to the end of the game

- Tic-Tac-Toe: fewer than 9! = 362,880 games — small enough to search completely.
- Chess: far too big to search completely → later: evaluation functions & pruning.

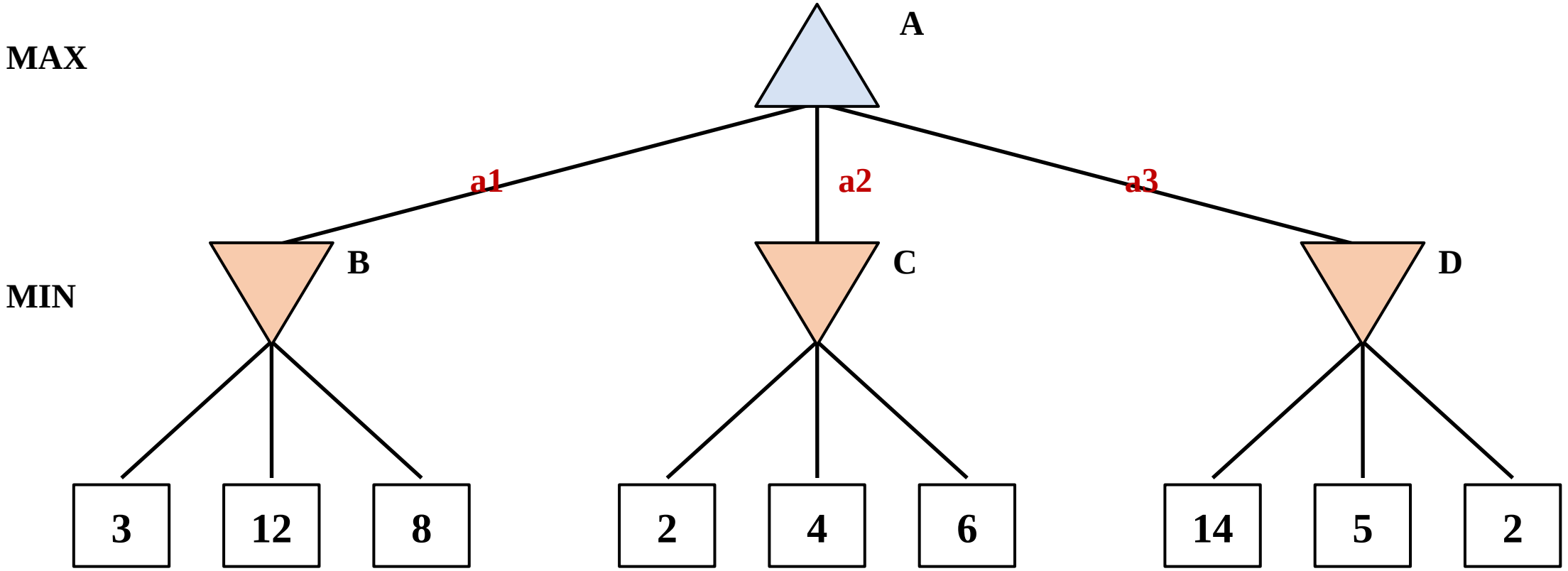
Minimax: The Idea

- **Assume both players play perfectly.**
- **MAX chooses the move with the highest value; MIN chooses the lowest.**
- **The minimax value of a node = the best score MAX can guarantee from that node.**
- **Values are computed bottom-up, from the end of the game to the current position:**

MINIMAX(s) =

Situation	Value
s is a terminal state (game over)	UTILITY(s)
It is MAX's turn	max over all moves a of MINIMAX(RESULT(s, a))
It is MIN's turn	min over all moves a of MINIMAX(RESULT(s, a))

Minimax: Step by Step (0/4)

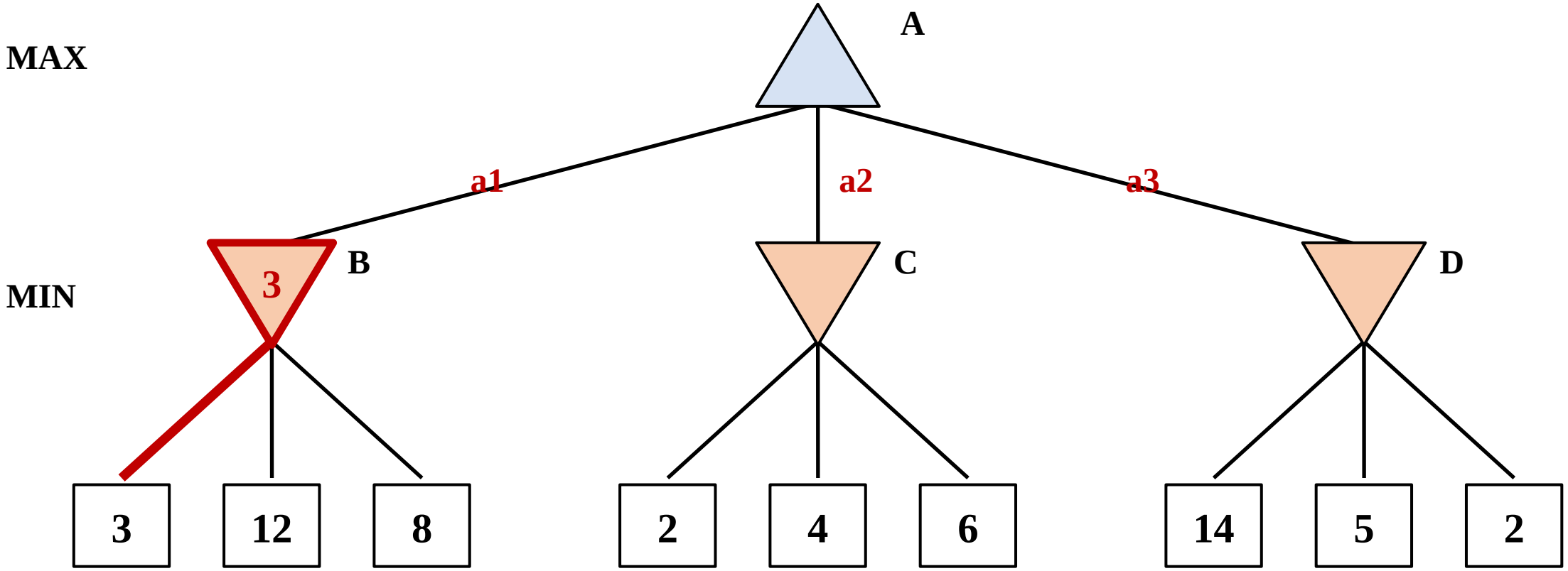


Start: only the leaves (end of game) have values

MAX moves first (3 choices: a1, a2, a3), then MIN replies. Numbers are utilities for MAX.

MAX (picks largest) **MIN (picks smallest)** **chosen move**

Minimax: Step by Step (1/4)

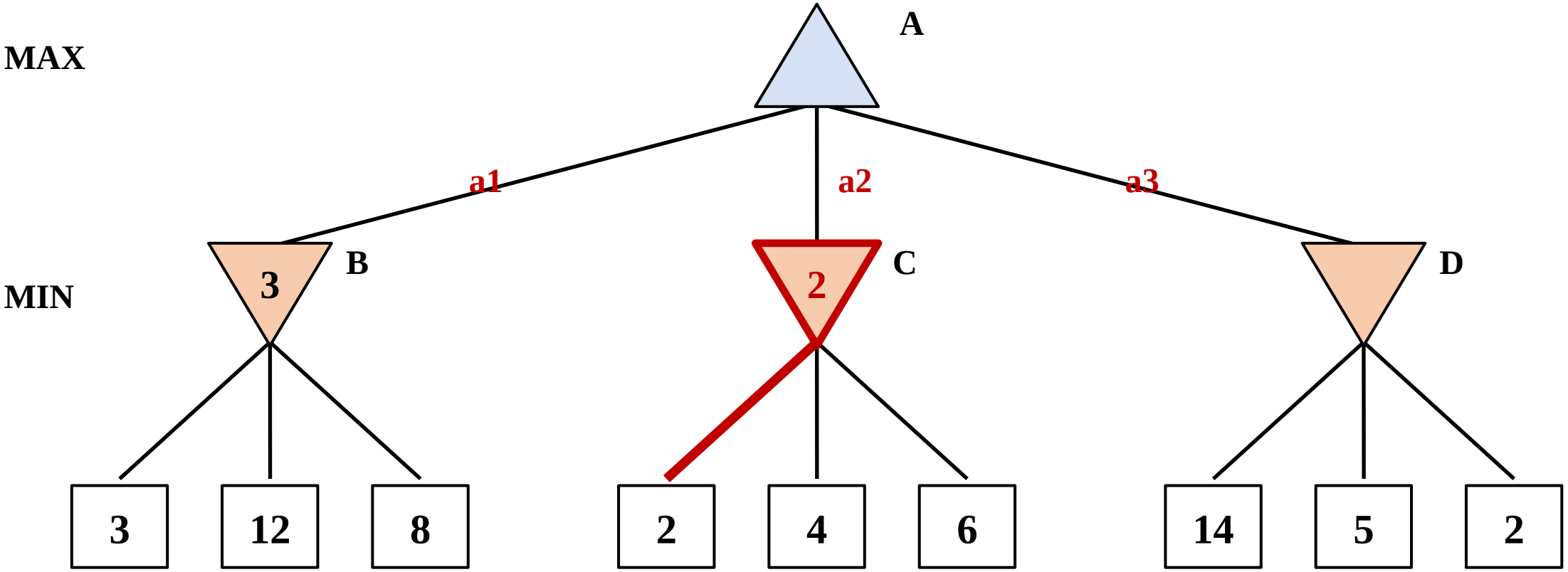


Step 1: B = min(3, 12, 8) = 3

At B it is MIN's turn, so MIN picks the smallest value: 3.

MAX (picks largest) MIN (picks smallest) chosen move

Minimax: Step by Step (2/4)

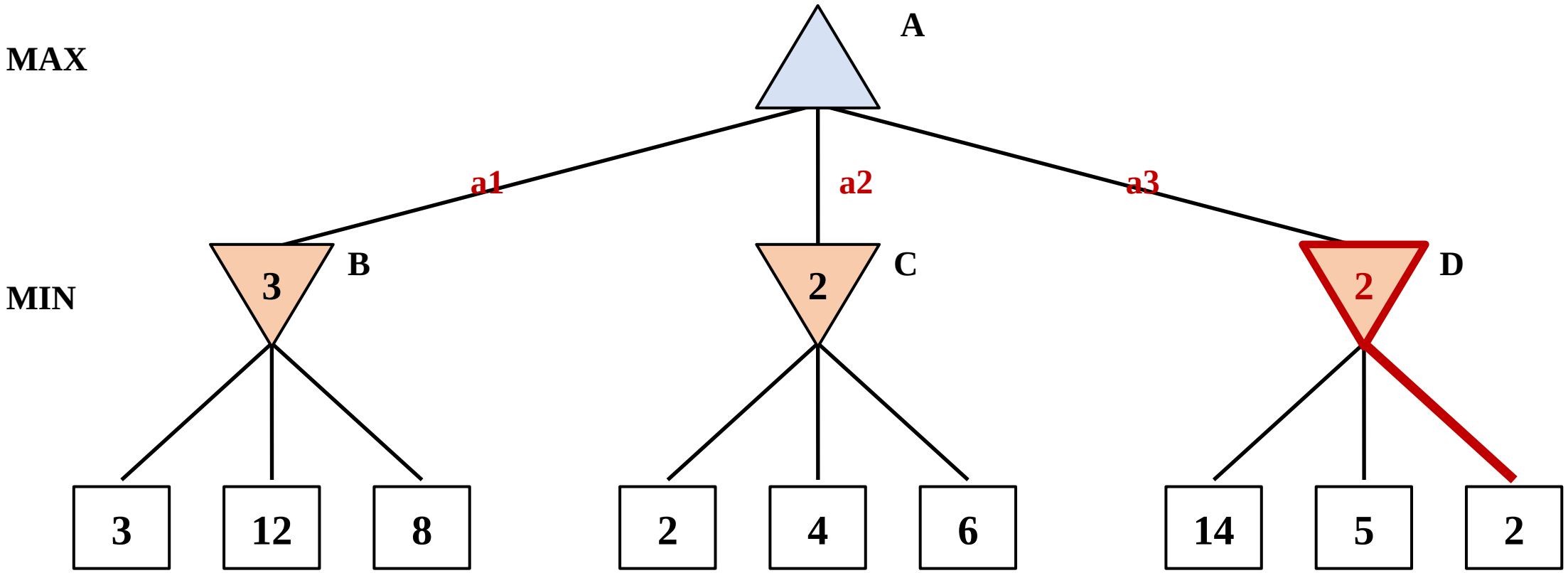


Step 2: C = min(2, 4, 6) = 2

MIN picks 2.

MAX (picks largest) MIN (picks smallest) chosen move

Minimax: Step by Step (3/4)

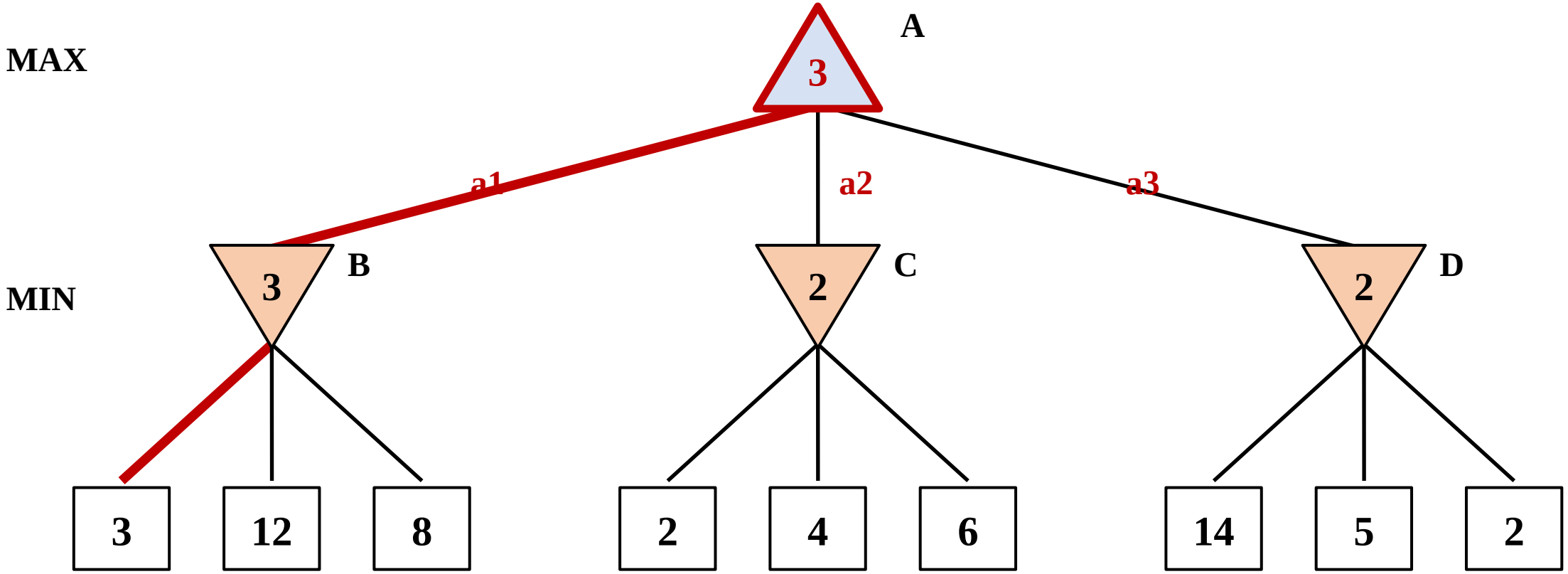


Step 3: D = min(14, 5, 2) = 2

MIN picks 2 — even though 14 is there, MIN will never allow it.

MAX (picks largest) **MIN (picks smallest)** **chosen move**

Minimax: Step by Step (4/4)

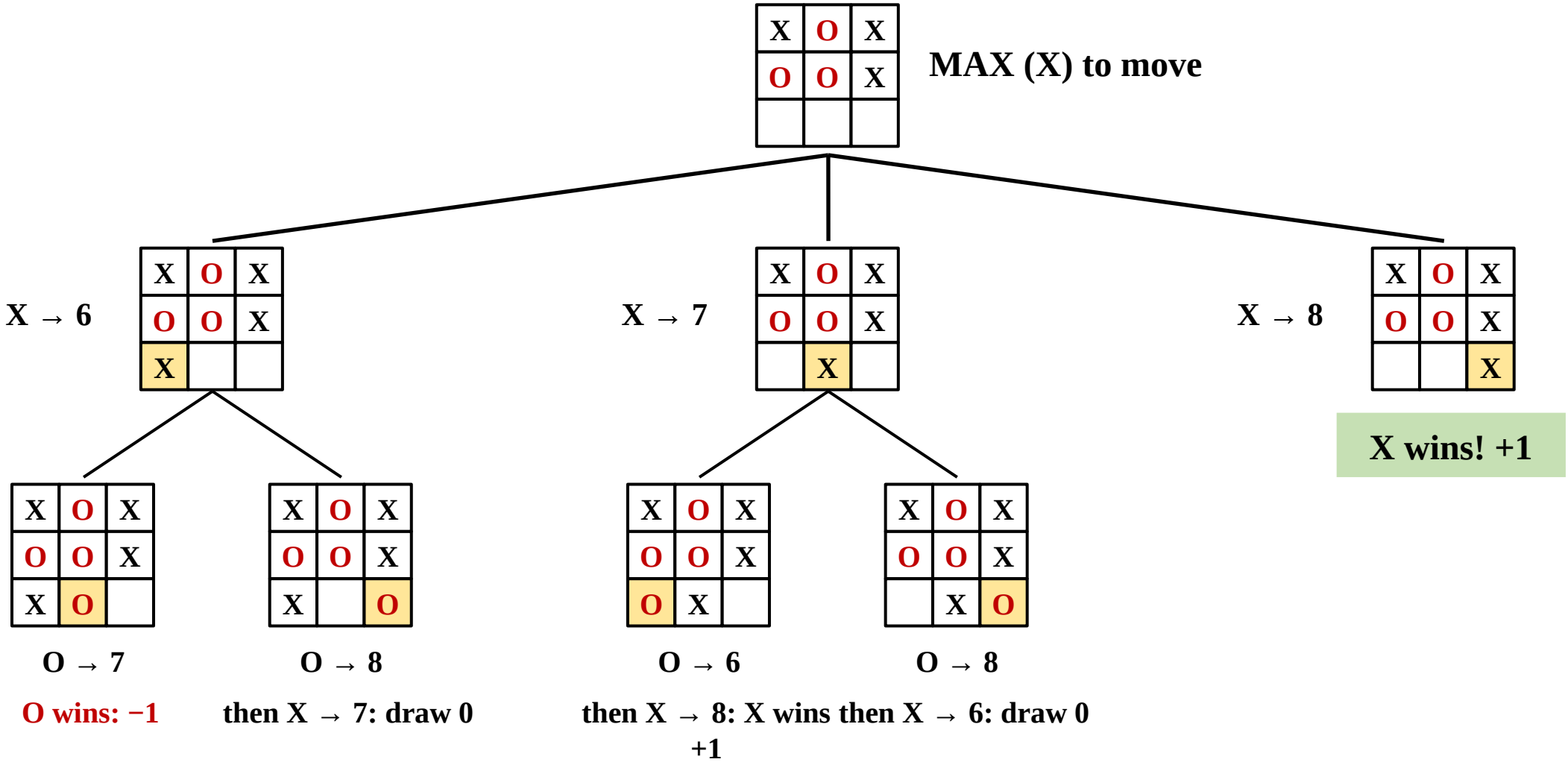


Step 4: A = max(3, 2, 2) = 3 → play a1 ✓

MAX picks the largest MIN value. The best guaranteed result is 3, by playing a1 (go to B).

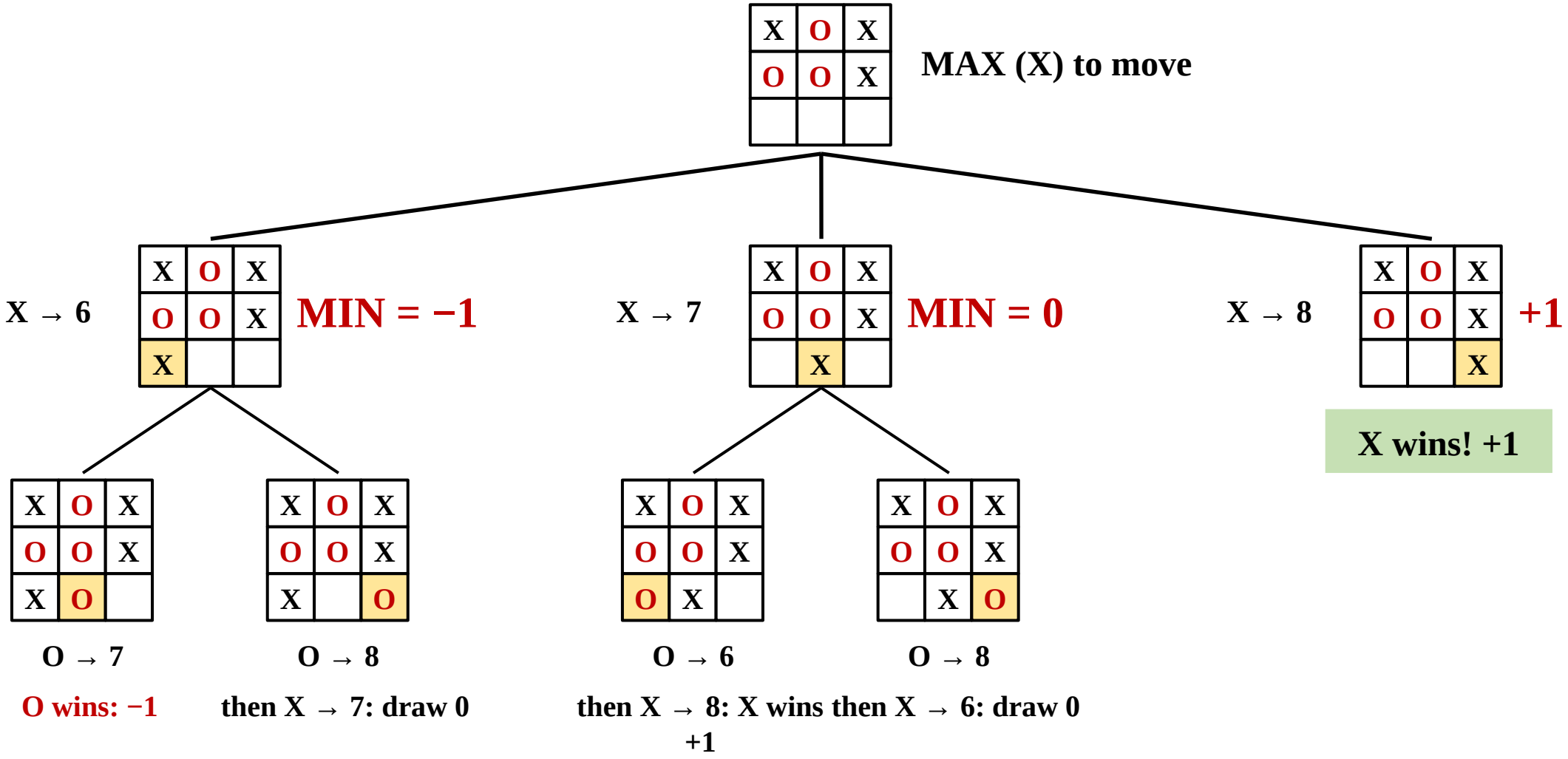
MAX (picks largest) MIN (picks smallest) chosen move

Tic-Tac-Toe Minimax (1/3): the game tree



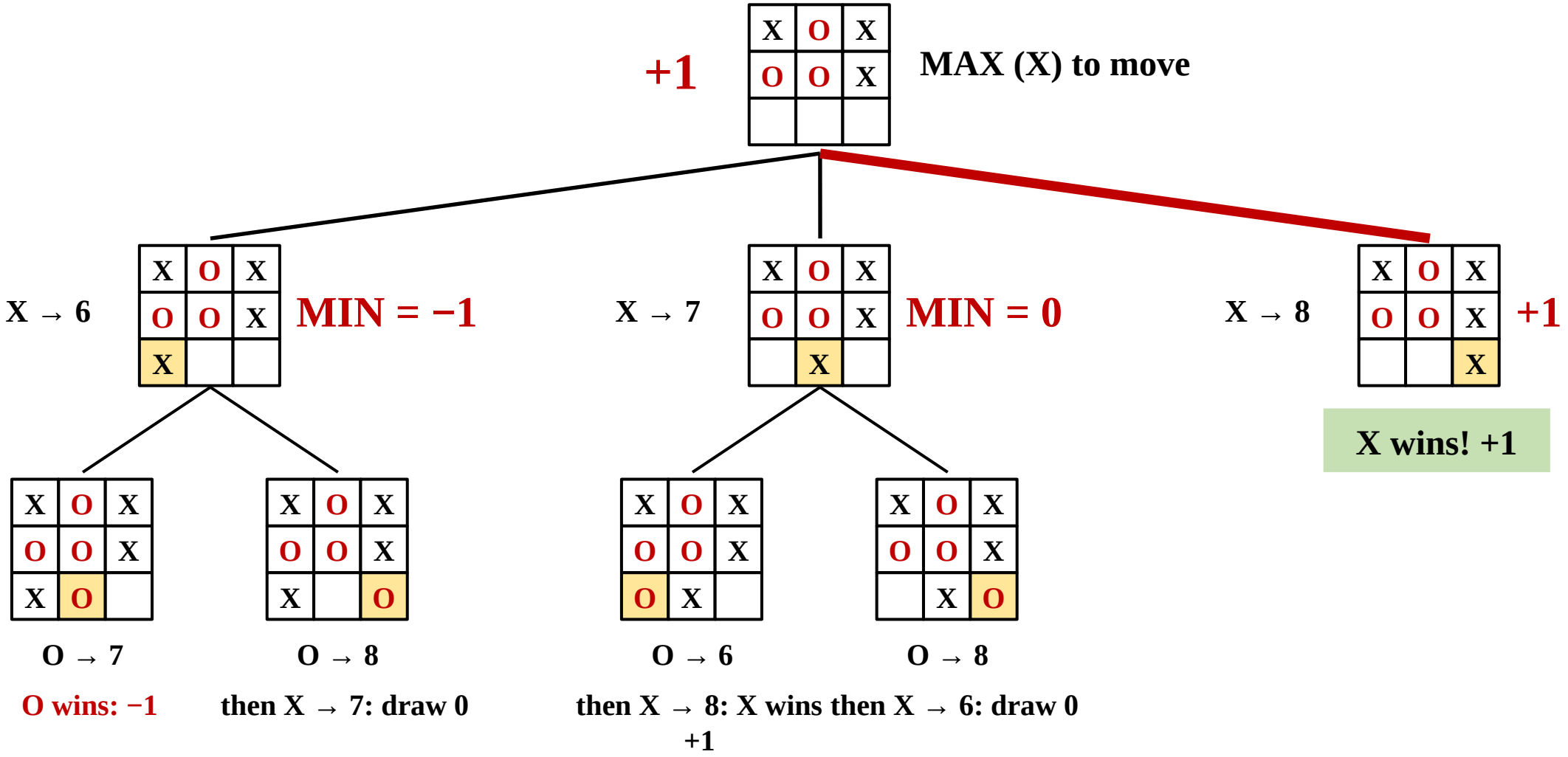
X (MAX) to move. Three empty squares: 6, 7, 8. Each branch is followed to the end of the game.

Tic-Tac-Toe Minimax (2/3): MIN values



If X plays 6, O wins by playing 7 → min = −1. If X plays 7, O's best is 8 → draw → min = 0. If X plays 8, X wins at once → +1.

Tic-Tac-Toe Minimax (3/3): MAX chooses



max(−1, 0, +1) = +1 → X plays square 8 and wins. Square numbers: 0 1 2 / 3 4 5 / 6 7 8.

The Minimax Algorithm

```
function MINIMAX-DECISION(state) returns an action
   return the action a with the largest MIN-VALUE(RESULT(state, a))

function MAX-VALUE(state):
   if TERMINAL-TEST(state): return UTILITY(state)
   v ← −∞
   for each action a: v ← max(v, MIN-VALUE(RESULT(state, a)))
   return v

function MIN-VALUE(state):
   if TERMINAL-TEST(state): return UTILITY(state)
   v ← +∞
   for each action a: v ← min(v, MAX-VALUE(RESULT(state, a)))
   return v
```

Properties of Minimax

Property	Minimax
Complete?	Yes, if the game tree is finite
Optimal?	Yes, against an optimal opponent
Time	$O(b^m)$ (b = legal moves, m = game length)
Space	O(bm) — depth-first

- Chess: b ≈ 35, m ≈ 100 → 35^{100} nodes — impossible to search completely.
- If MIN plays badly, MAX does at least as well as the minimax value.
- Next class: Alpha–Beta pruning and evaluation functions make game search practical.

Tic-Tac-Toe Minimax in Python

```
LINES = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]

def winner(b):
    for i, j, k in LINES:
        if b[i] != ' ' and b[i] == b[j] == b[k]: return b[i]
    return None

def minimax(b, player):            # X = MAX, O = MIN
    w = winner(b)
    if w == 'X': return 1
    if w == 'O': return -1
    if ' ' not in b: return 0      # draw
    other = 'O' if player == 'X' else 'X'
    values = [minimax(b[:i] + player + b[i+1:], other) for i in range(9) if b[i] == ' ']
    return max(values) if player == 'X' else min(values)

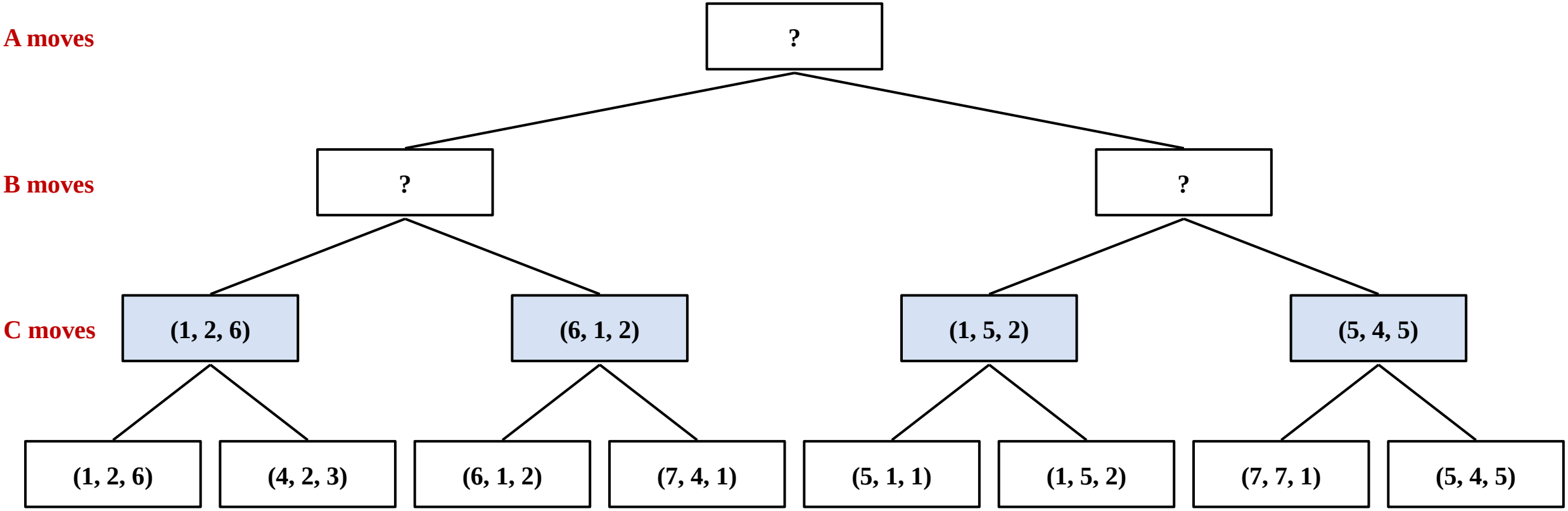
def best_move(b):                  # X to move
    moves = [i for i in range(9) if b[i] == ' ']
    return max(moves, key=lambda i: minimax(b[:i] + 'X' + b[i+1:], 'O'))

print(best_move('XOXOOX   '))     # 8
print(minimax(' ' * 9, 'X'))     # 0  (perfect play = draw)
```

Optimal Decisions in Multiplayer Games

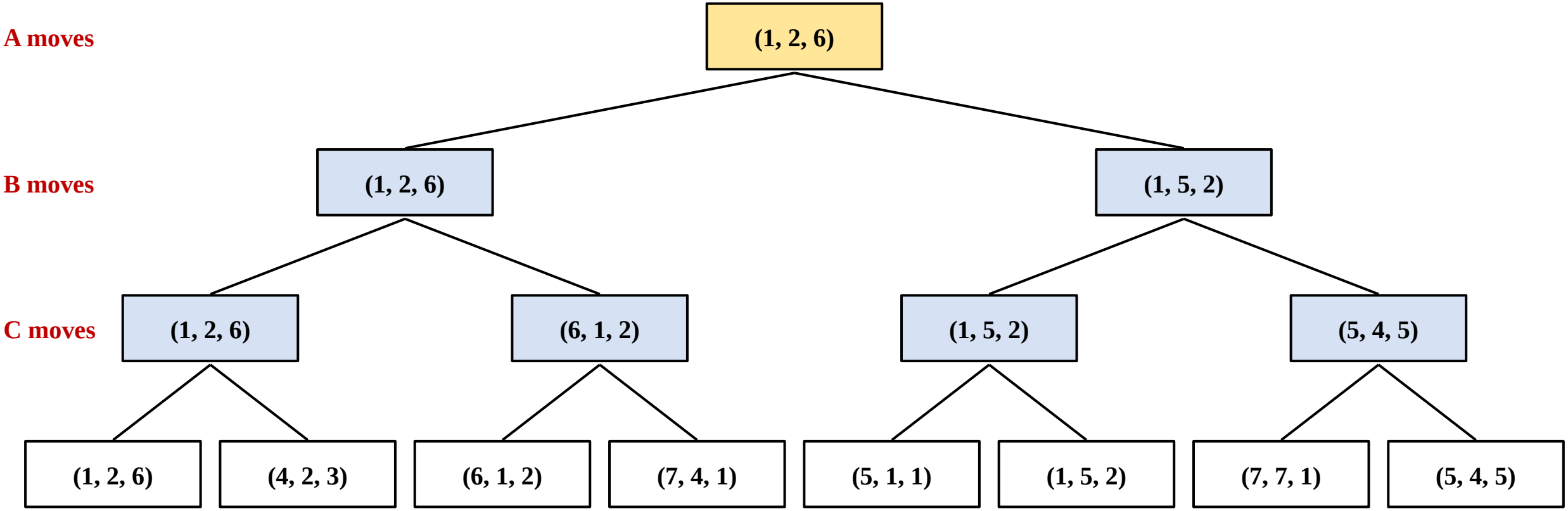
- **With 3 or more players (e.g. Ludo), one number is not enough.**
- **Each state has a vector of utilities: (u_A, u_B, u_C) — one score per player.**
- **At each node, the player whose turn it is picks the child with the highest value for itself.**
- **Alliances can form: two weaker players may team up against the leader, then break up later.**
- **Two-player zero-sum games are the special case where u_MIN = −u_MAX → plain minimax.**

Multiplayer Example: Step by Step (1/2)



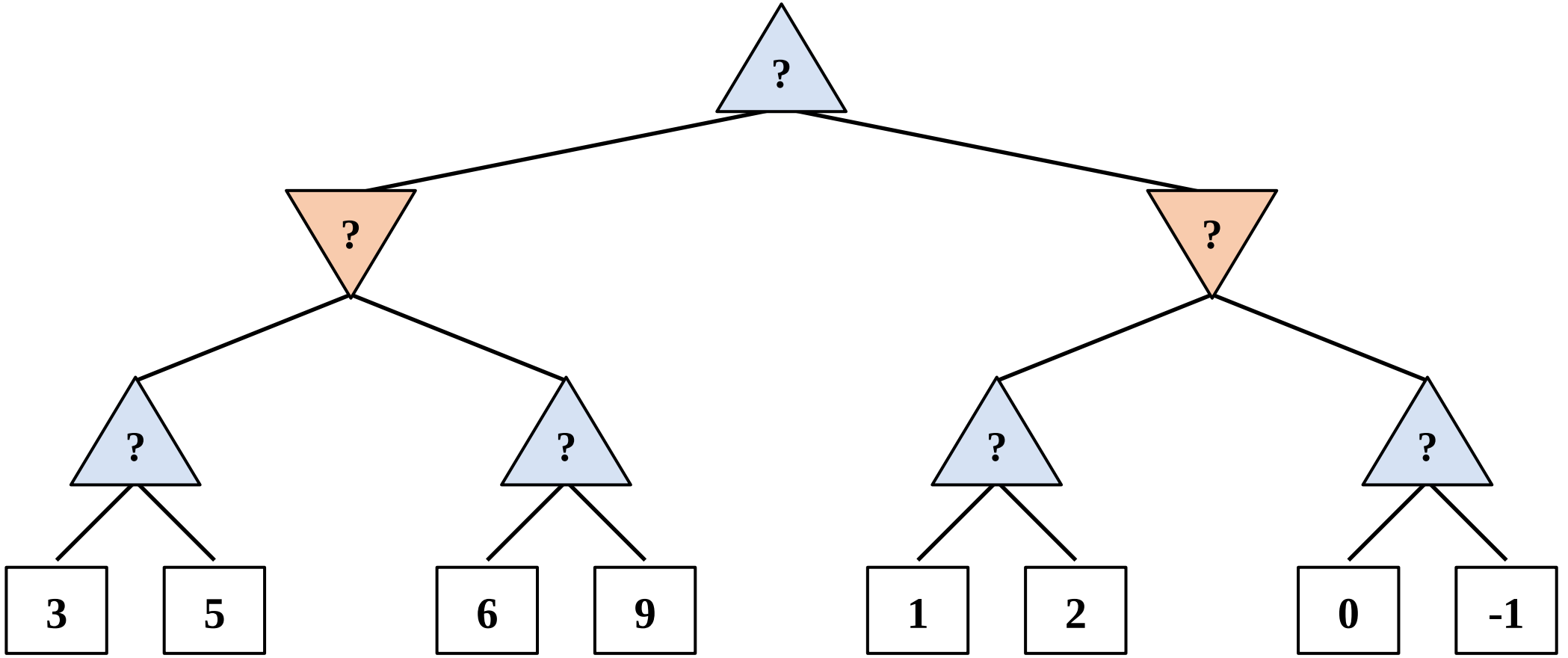
C chooses the child with the largest 3rd value: (1,2,6), (6,1,2), (1,5,2), (5,4,5).

Multiplayer Example: Step by Step (2/2)



B chooses the largest 2nd value: (1,2,6) and (1,5,2). A chooses the largest 1st value: tie (1 = 1) → (1, 2, 6).

Class Exercise (5 minutes)



Fill in every ? with its minimax value. Which move should MAX play at the top?

MAX (picks largest)

MIN (picks smallest)

chosen move

Quick Check

- 1. What does zero-sum mean?
- 2. What gives the score of a finished game?
- 3. At a MIN node, which child value is chosen?
- 4. What are the time and space complexity of minimax?
- 5. How do values work in a game with three players?

Summary

- **Adversarial search = search when an opponent works against us.**
- **A game = initial state, PLAYER, ACTIONS, RESULT, TERMINAL-TEST, UTILITY.**
- **Minimax: MAX takes the highest value, MIN the lowest; values are computed bottom-up.**
- **Minimax is complete and optimal, but takes $O(b^m)$ time — too slow for chess.**
- **Multiplayer games use a vector of utilities; each player maximises its own value.**
- **Reading: Russell & Norvig, AIMA, 5.1–5.2.**
- **Next class: Alpha–Beta pruning and evaluation functions.**

Thank You

Any questions?